

HMM: Inference

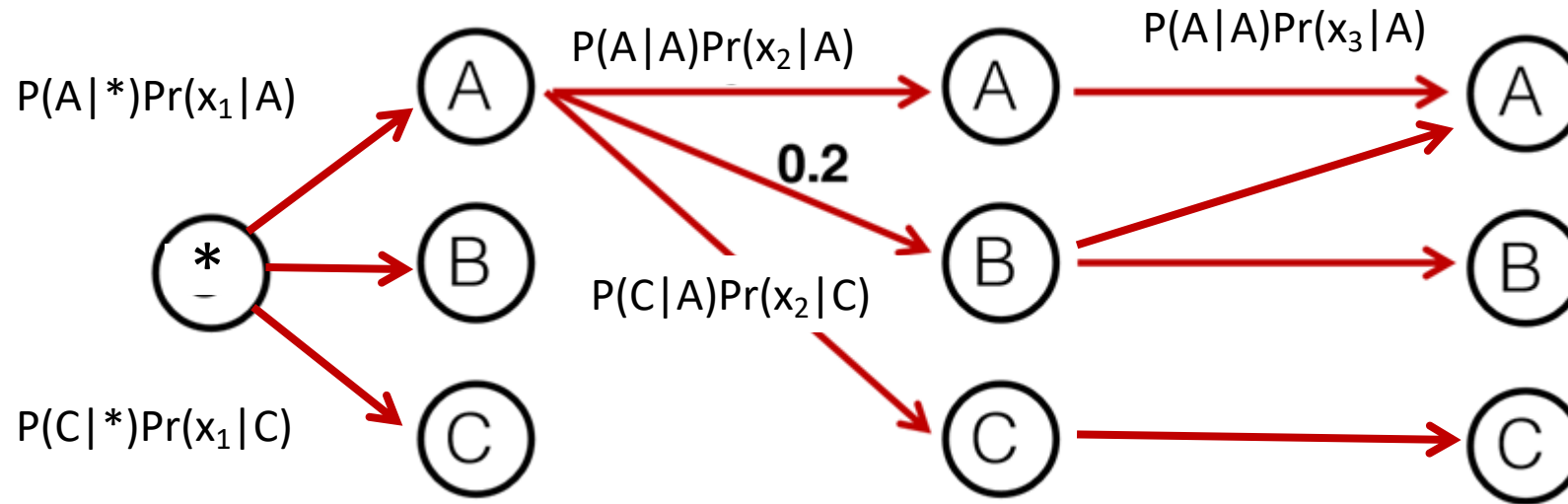
Brute Force Search is Hopelessly Inefficient

Problem: for an input $x_1 \dots x_n$, find

$$\arg \max_{y_1 \dots y_{n+1}} p(x_1 \dots x_n, y_1 \dots y_{n+1})$$

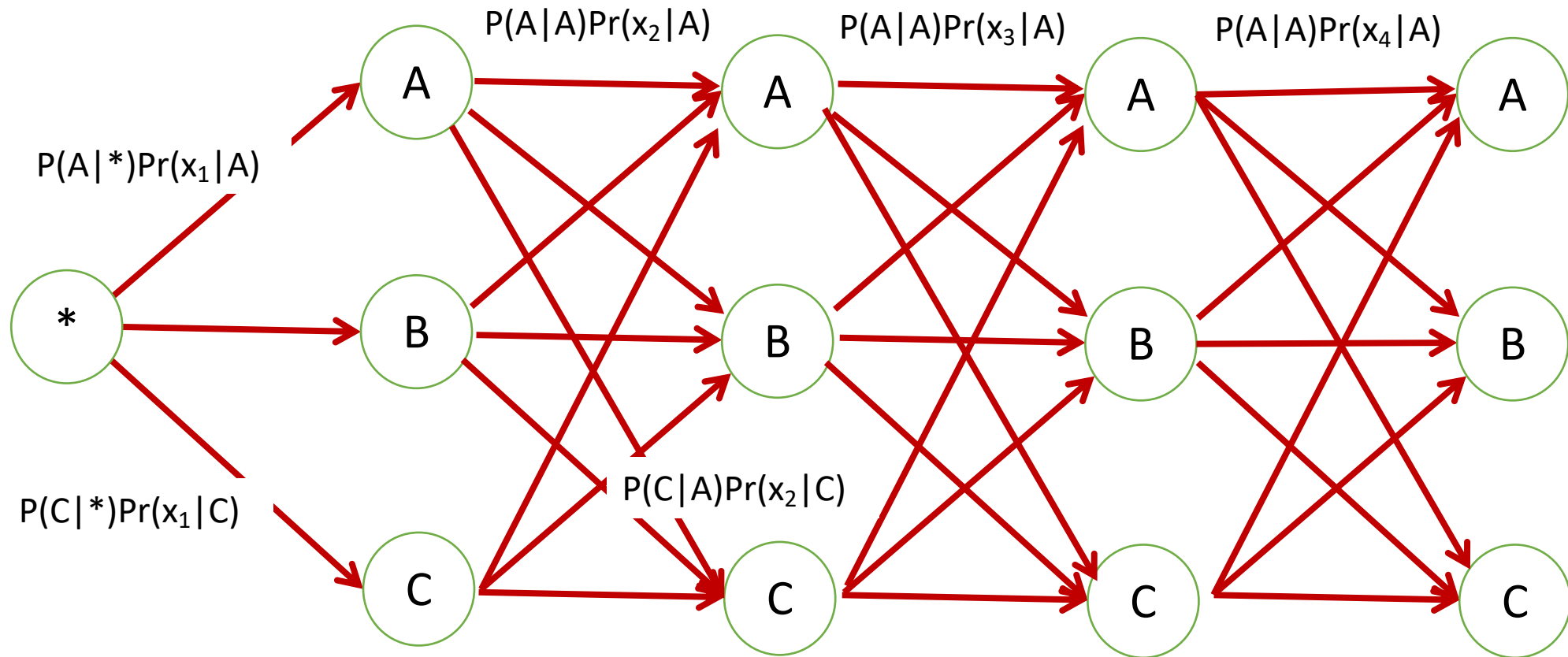
where the $\arg \max$ is taken over all sequences $y_1 \dots y_{n+1}$ such that $y_i \in \mathcal{S}$ for $i = 1 \dots n$, and $y_{n+1} = \text{STOP}$.

HMM Inference: Graphic Presentation

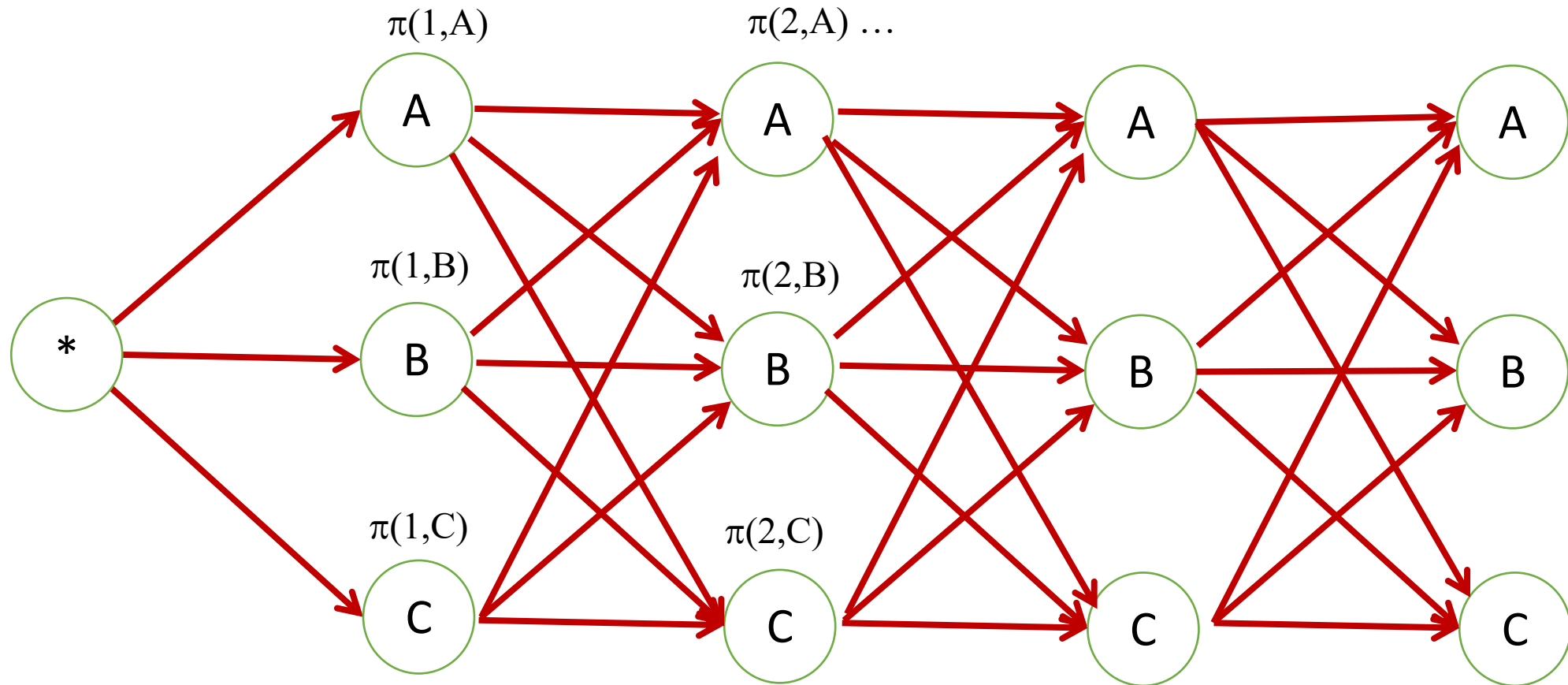


- Finding the most likely assignment for $x_1, \dots, x_n$ that ends with a certain state s is equivalent to finding the maximum weighted path from $*$ to the state s in the n -th layer
 - The weight of a path is here the product of the weights of its edges
- a dynamic programming approach solves the inference problem

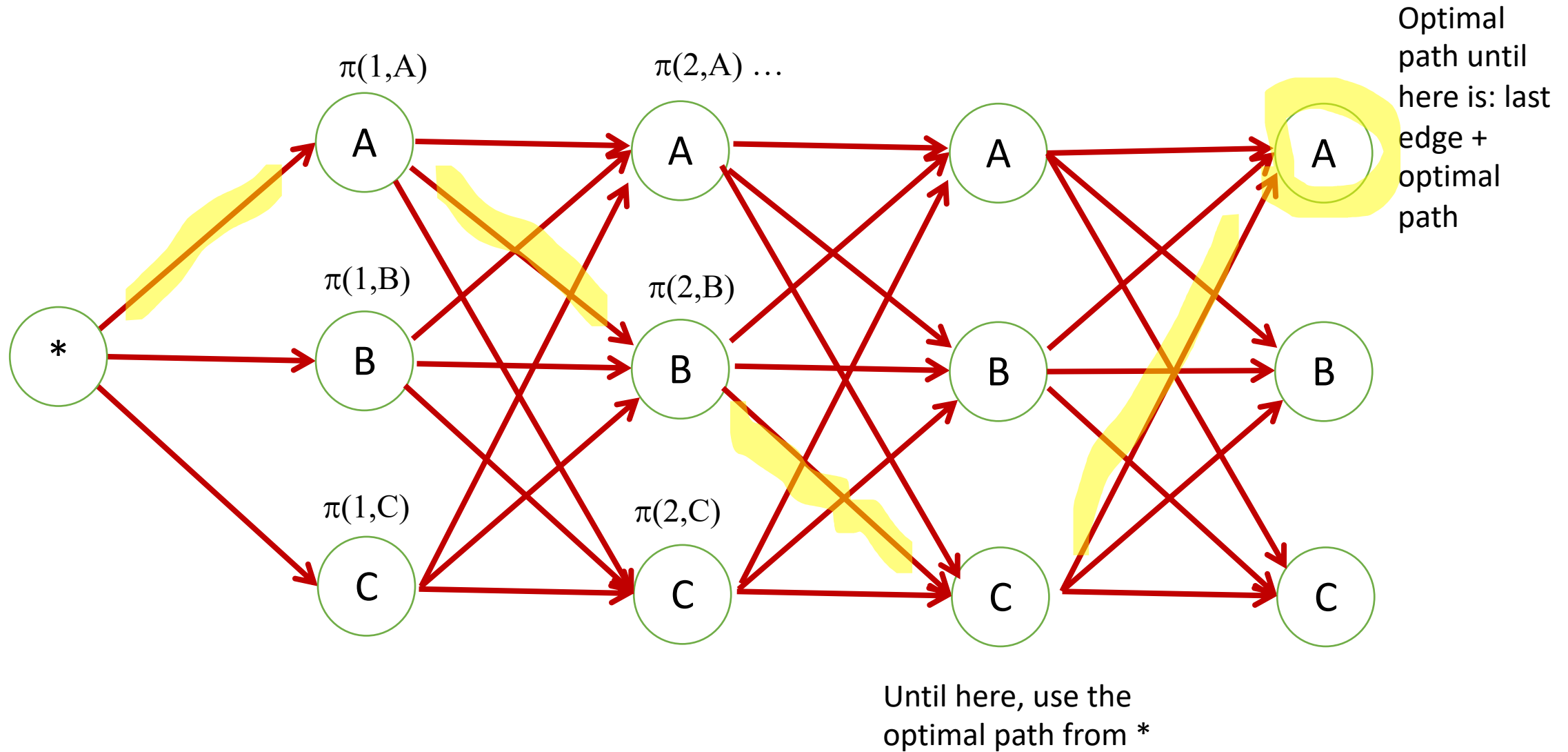
HMM Inference: Graphic Presentation



HMM Inference: Graphic Presentation



HMM Inference: Dynamic Programming



HMM: Inference

The Viterbi Algorithm

Problem: for an input $x_1 \dots x_n$, find

$$\arg \max_{y_1 \dots y_{n+1}} p(x_1 \dots x_n, y_1 \dots y_{n+1})$$

where the $\arg \max$ is taken over all sequences $y_1 \dots y_{n+1}$ such that $y_i \in \mathcal{S}$ for $i = 1 \dots n$, and $y_{n+1} = \text{STOP}$.

We assume that p again takes the form

$$p(x_1 \dots x_n, y_1 \dots y_{n+1}) = \prod_{i=1}^{n+1} q(y_i | y_{i-2}, y_{i-1}) \prod_{i=1}^n e(x_i | y_i)$$

Recall that we have assumed in this definition that $y_0 = y_{-1} = *$, and $y_{n+1} = \text{STOP}$.

HMM: Inference

The Viterbi Algorithm

- ▶ Define n to be the length of the sentence
- ▶ Define S_k for $k = -1 \dots n$ to be the set of possible tags at position k :

$$S_{-1} = S_0 = \{*\}$$
$$S_k = S \quad \text{for } k \in \{1 \dots n\}$$

- ▶ Define

$$r(y_{-1}, y_0, y_1, \dots, y_k) = \prod_{i=1}^k q(y_i | y_{i-2}, y_{i-1}) \prod_{i=1}^k e(x_i | y_i)$$

- ▶ Define a dynamic programming table

$$\pi(k, u, v) = \text{maximum probability of a tag sequence} \\ \text{ending in tags } u, v \text{ at position } k$$

that is,

$$\pi(k, u, v) = \max_{\langle y_{-1}, y_0, y_1, \dots, y_k \rangle : y_{k-1}=u, y_k=v} r(y_{-1}, y_0, y_1 \dots y_k)$$

HMM: Inference

•,

A Recursive Definition

Base case:

$$\pi(0, *, *) = 1$$

Recursive definition:

For any $k \in \{1 \dots n\}$, for any $u \in \mathcal{S}_{k-1}$ and $v \in \mathcal{S}_k$:

$$\pi(k, u, v) = \max_{w \in \mathcal{S}_{k-2}} (\pi(k-1, w, u) \times q(v|w, u) \times e(x_k|v))$$

HMM: Inference with the Viterbi Algorithm

The Viterbi Algorithm

Input: a sentence $x_1 \dots x_n$, parameters $q(s|u, v)$ and $e(x|s)$.

Initialization: Set $\pi(0, *, *) = 1$

Definition: $\mathcal{S}_{-1} = \mathcal{S}_0 = \{*\}$, $\mathcal{S}_k = \mathcal{S}$ for $k \in \{1 \dots n\}$

Algorithm:

- ▶ For $k = 1 \dots n$,

- ▶ For $u \in \mathcal{S}_{k-1}$, $v \in \mathcal{S}_k$,

$$\pi(k, u, v) = \max_{w \in \mathcal{S}_{k-2}} (\pi(k-1, w, u) \times q(v|w, u) \times e(x_k|v))$$

- ▶ **Return** $\max_{u \in \mathcal{S}_{n-1}, v \in \mathcal{S}_n} (\pi(n, u, v) \times q(\text{STOP}|u, v))$

HMM: Inference with the Viterbi Algorithm

The Viterbi Algorithm with Backpointers

Input: a sentence $x_1 \dots x_n$, parameters $q(s|u, v)$ and $e(x|s)$.

Initialization: Set $\pi(0, *, *) = 1$

Definition: $\mathcal{S}_{-1} = \mathcal{S}_0 = \{*\}$, $\mathcal{S}_k = \mathcal{S}$ for $k \in \{1 \dots n\}$

Algorithm:

- ▶ For $k = 1 \dots n$,
 - ▶ For $u \in \mathcal{S}_{k-1}$, $v \in \mathcal{S}_k$,
$$\pi(k, u, v) = \max_{w \in \mathcal{S}_{k-2}} (\pi(k-1, w, u) \times q(v|w, u) \times e(x_k|v))$$
$$bp(k, u, v) = \arg \max_{w \in \mathcal{S}_{k-2}} (\pi(k-1, w, u) \times q(v|w, u) \times e(x_k|v))$$
- ▶ Set $(y_{n-1}, y_n) = \arg \max_{(u, v)} (\pi(n, u, v) \times q(\text{STOP}|u, v))$
- ▶ For $k = (n-2) \dots 1$, $y_k = bp(k+2, y_{k+1}, y_{k+2})$
- ▶ **Return** the tag sequence $y_1 \dots y_n$

HMM: Inference with the Viterbi Algorithm

The Viterbi Algorithm: Running Time

- ▶ $O(n|\mathcal{S}|^3)$ time to calculate $q(s|u, v) \times e(x_k|s)$ for all k, s, u, v .
- ▶ $n|\mathcal{S}|^2$ entries in π to be filled in.
- ▶ $O(|\mathcal{S}|)$ time to fill in one entry